CS2040S Cheatsheet

Order of Growth

Properties

Let $T(n) = O(f(n))$ and $S(n) = O(g(n))$

- **Addition:** $T(n) + S(n) = O(f(n) + g(n))$
- **Multiplication:** $T(n) \cdot S(n) = O(f(n) \cdot g(n))$
- **Composition:** $T(n) \circ S(n) = O(f(n) \circ g(n))$
  - *Only if both functions are increasing*
- **Max:** $\max(f(n), g(n)) \leq f(n) + g(n)$

Space Complexity

- Take the **max space** taken at any point of time
- For example:

```
int logN = Math.log(n);

for (int i = 0; i < n; i++) {
    int[] newArray = new int[logN];
}
```

  - Takes up $O(\log(n))$ space instead of $O(n \cdot \log(n))$

Sorting Algorithms

Insertion Sort

```
void insertionArray(int arr[], int n) {
    For each element from the first to last element
    Keep trying to shift that element if it's smaller
    than the element to the left
}
```

- **Invariant:** At the start of the ith iteration, the sub-array $arr[0, (i-1)]$ is in sorted order
- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(1)$

Selection Sort

```
void insertionSort(int arr[], int n){
    For each position i from 0 to n - 1
        Find minimum element in sub-array arr[i..(n-1)].
        Set the element at position i in arr to the minimum
        element found.
}
```

- At the beginning of the ith iteration, the sub-array $arr[0 \ldots (i-1)]$ is the same as the sub-array $sorted[0 \ldots (i-1)]$
- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(1)$

Merge Sort

```
void mergeSort(int arr[], int left, int right){
    if left + 1 == right:
        return;
    mid = (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid, right);
    output = new int[right - left];
    merge(arr, left, right, output);
    copy from output to arr[left, right)
}
```

- At the beginning of the ith iteration, the sub-array $arr[0 \ldots (i-1)]$ is the same as the sub-array $sorted[0 \ldots (i-1)]$
- **Time Complexity:** $O(n \cdot \log(n))$
- **Space Complexity:** $O(n)$

Quick Sort

```
void quickSort(int arr[], int left, int right){
    if left + 1 == right:
        return; // empty array do nothing

    // partition array based on the first element of array
    pivot_idx = partition(arr, left);

    Quicksort(arr, left, pivot_idx);
    Quicksort(arr, pivot_idx + 1, right);
}
```

- At the beginning of each iteration, the elements to the left of a certain index i are all less than or equal to the pivot ($arr[i]$) and vice versa
- **Time Complexity:** Depends on choice of pivot
  - Worst case: $O(n^2)$ first/last pivot
  - Worst case: $O(n \cdot \log(n))$ median element
  - Worst case: ∞ random element
- **Space Complexity:** $O(n)$

Binary Search Trees

General

- a BST is either empty, or a node pointing to 2 BSTs
- Balanced tree: $h = \log_2 n$
- For a full binary tree of size n, $\exists k \in \mathbb{Z}^+$ s.t. $n = 2^k - 1$

Runtime for Operations

- *Lookup:* $O(n)$ when every node has a single child
  - If not, can be $O(h)$, where h is the height
- *Insert:* $O(h)$
- *Delete:* $O(h)$
  - Case 1: No children - remove the node
  - Case 2: 1 child - remove the node, connect parent to child
  - Case 3: 2 children - delete the successor; replace node with successor

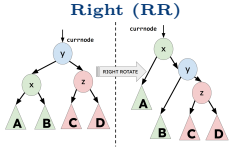
AVL Trees

General

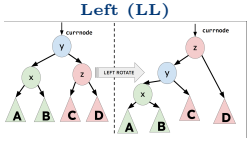
- Invariant: **Height balanced** (maintained with rotations)
  - $\Leftrightarrow |v.left.height - v.right.height| \leq 1$

Rotations

Right (RR)



Left (LL)



Terminology

- **Left/Right Heavy:** Height of left/right child node is taller than height of right/left child node
  - Perform a right/left-rotation

AVL Rebalancing Cases

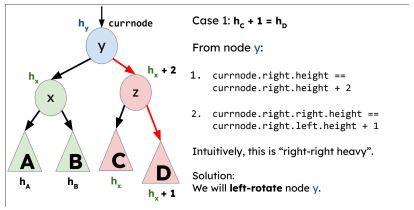
Check balance factor: $bf = height(L) - height(R)$. *Rebalance if* $|bf| > 1$

1. Left-Left (LL) Heavy

- **Condition:** y is left-heavy, and $y.left$ is left-heavy
- **Fix:** Single **Right Rotation** on y

2. Right-Right (RR) Heavy

- **Condition:** y is right-heavy, and $y.right$ is right-heavy
- **Fix:** Single **Left Rotation** on y

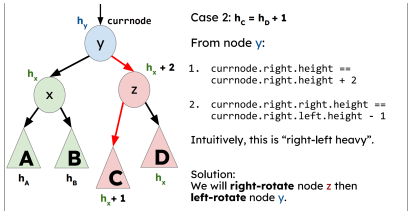


3. Left-Right (LR) Heavy

- **Condition:** y is left-heavy, but $y.left$ is right-heavy
- **Fix: Double Rotation**: Left-rotate $y.left$, then Right-rotate y

4. Right-Left (RL) Heavy

- **Condition:** y is right-heavy, but $y.right$ is left-heavy
- **Fix: Double Rotation**: Right-rotate $y.right$, then Left-rotate y



Tree Augmentation Summary

- **Size Augment:** Store subtree size to support $Rank(k)$ and $Select(i)$ in $O(h)$
- **Rank:** Number of elements smaller than k
- **Select:** Find key with a given rank
- **Maintenance:** Update $height$ and $size$ as recursion unwinds

AVL Deletion

1. BST Deletion Phase

- **Leaf:** Just remove the node
- **1 Child:** Remove node and connect parent to the child
- **2 Children:** Find successor n (min of right subtree), swap key with n, and delete n (which now has ≤ 1 child)

2. Rebalancing Phase

As recursion unwinds, update heights and check $bf = h_L - h_R$. *If* $|bf| > 1$, *rebalance at the first unbalanced node* y

Case: Deleted from Right Subtree ($bf = 2$)

- **LL Heavy:** If $h(y.L.L) \geq h(y.L.R)$, perform **Right Rotation** on y
- **LR Heavy:** If $h(y.L.L) < h(y.L.R)$, perform **Left-Rotate** $y.L$, then **Right-Rotate** y

Case: Deleted from Left Subtree ($bf = -2$)

- **RR Heavy:** If $h(y.R.R) \geq h(y.R.L)$, perform **Left Rotation** on y
- **RL Heavy:** If $h(y.R.R) < h(y.R.L)$, perform **Right-Rotate** $y.R$, then **Left-Rotate** y

Key Difference from Insertion

- **Propagation:** While insertion requires at most one rebalance, **deletion can trigger rebalancing at multiple levels** up to the root.
- **Complexity:** $O(\log n)$ for all cases

Hash Tables

- Insert, delete, and lookup should run in expected $O(1)$ time, and size in worst-case $O(1)$ time.

Hash Function

- **Load Factor** (α): n/m, representing the expected length of a chain.
- **Criteria for a Good Hash Function:** Must evaluate deterministically, look random for new inputs, and execute efficiently.
- **Simple Uniform Hashing Assumption (SUHA):** Assumes each new key picks any table index uniformly at random, ensuring keys are distributed evenly. If $n/m = O(1)$, expected runtime of operations is $O(1)$.
- **Division Method:** $h(k) = k \bmod m$ (choose prime m, avoid $m = 2^x$)
- **Multiplication Method:** $h(k) = (Ak) \bmod 2^w \gg (w - r)$

Collision Resolution

- **Separate Chaining**
  - Each slot in the table acts as a head for a linked list.
  - **Worst-Case Runtime:** $O(n)$ for insert, lookup, and delete if all items collide into a single list.
  - **Expected Runtime (SUHA):** $O(n/m)$ for insert, lookup, and delete.
- **Open Addressing (Linear Probing)**
  - Stores key-value pairs directly within the table array itself.
  - **Mechanism:** If an index is occupied, sequentially try the next slot until an empty one is found.
  - **Worst-Case Runtime:** $O(n)$ for insert, lookup, and delete.
  - **Expected Runtime (SUHA):** $O(1)$ for insert, lookup, and delete.
  - **Deletion Constraints:** Simply removing an item breaks the search "run" for subsequent keys.
  - **Deletion Fix (Shifting):** When removing at index i, check subsequent items e. If i is within e's target interval, move e to index i.
  - **Interval Cases:**
    - Case 1 (No wrap): If $h(e) \leq$ index, the interval is $[h(e), \text{index}]$.
    - Case 2 (Wraparound): If $h(e) >$ index, the interval is $[h(e), m-1]$ and $[0, \text{index}]$.
- **Double Hashing**
  - $(h(k, i) = f(k) + i \cdot g(k)) \bmod m$
    - k: The key that is being inserted or searched for
    - i: The probe attempt number (starts at 0, then $1, 2, 3, \ldots$)
    - $f(k)$: The primary hash function to give your starting idx
    - $g(k)$: The secondary hash function to give step size
    - m: Total size of hash table

Advanced Trees

(a, b)-trees & B-Trees

- **B-Tree:** A $(B, 2B)$-tree is an (a, b)-tree where $a = B$ and $b = 2B$
- **Rules:**

	# keys		# children	
node type	min	max	min	max
root	1	$b-1$	2	b
internal	$a-1$	$b-1$	a	b
leaf	$a-1$	$b-1$	0	0

  - (a, b)-child policy: $2 \leq a \leq (b+1)/2$
  - Internal node has 1 more child than its number of keys
  - All leaf nodes must be at the **same depth** from the root
- **Heights:** Max $O(\log_a n) + 1$, Min $O(\log_b n)$
- **Operations:** *Search, Insert, Delete* are all $O(\log n)$
  - **Split:** When a node has too many children (move median key to parent)
  - **Merge/Share:** When a node becomes empty during deletion

Trie

- **Search, Insert:** $O(L)$ for a string of length L
- **Space:** $O(\text{size of text} \cdot \text{overhead})$

kd-Tree

- Stores geometric data (points in an (x, y) plane)
- Alternates splitting (partitioning) via x and y coordinates

- **Construct:** $O(n \log n)$
- **Search:** $O(h)$
- **SearchMin:** $T(n) = 2T(n/4) + O(1) \implies O(\sqrt{n})$

Merkle Trees

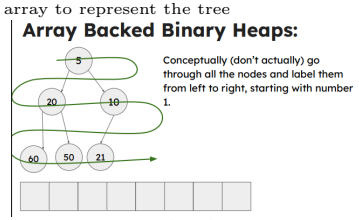
- Binary tree - nodes augmented with a hash of their children
- Same root value $\implies$ identical tree

Merkle Tree

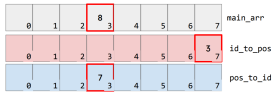
Binary Heaps

General

- Visualised in the form of a complete binary tree
  - Every node has either 0, 1, or 2 children
  - Every level **must be full** except the last level
  - Last Level must be filled from left to right
- Priorities of children are $\geq$ priority of parents (min heap)
- Use array to represent the tree

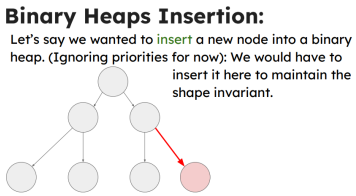


- Make sure array is 1-indexed
- Tree traversal in array
  - Go to left child: $idx \times 2$
  - Go to right child: $idx \times 2 + 1$
  - Go to parent: $\lfloor idx \div 2 \rfloor$
- To store values, we use 2 more arrays
  - id_to_pos
  - pos_to_id



  - For swaps
    1. Swap in main array and store idx
    2. Find id from pos_to_id array
    3. Swap in the pos_to_id array
    4. Use the id from step 2 to make the swap in id_to_pos array

Insertion



1. Insert at the end of the heap
2. Keep swapping with parent node if parent node's priority is greater than current node

Deletion

1. Swap the node you want to delete with the last node
2. Delete the last node (the node you want to delete)
3. Compare the newly swapped node with its new parent
   - If it violates heap property, bubble up
   - Otherwise, bubble down

Extract Min

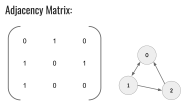
1. Swap root node with last node
2. Remove the last node and return
3. Bubble the root node down if it violates binary heap invariant with the smaller priority child

Heapify (Bubble Down / Sift Down)

1. Shift all elements by 1 to make it 1-indexed
2. Compare the current node's priority with its children for idx from $(size \div 2)$ to 1
3. If it violates the binary heap invariant, swap it with the highest-priority child (e.g., the smaller child in a Min-Heap)
4. Repeat this process on the node's new position until the invariant is restored or it becomes a leaf node

Graphs

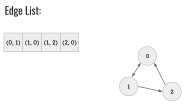
Adjacency Matrix



- arr[i][j] gives the weight (1 if unweighted) of the edge from i to j (or 0 if no edge)

Adjacency List

Edge List



- If edges are weighted, store as triplets $(u, v, weight)$

	AM	AL	EL
is_adj	$O(1)$	$O(\deg i)$	$O(m)$
get_neigh	$O(n)$	$O(1)$	$O(m)$
in_deg	$O(n)$	$O(n+m)$	$O(m)$
out_deg	$O(n)$	$O(1)$	$O(m)$
space	$O(n^2)$	$O(n+m)$	$O(m)$

Searching

Breadth-First Search (BFS)

- Uses a queue to explore the graph level by level
- Time Complexity: $O(n+m)$
- Space Complexity: $O(n)$

Depth-First Search (DFS)

- Uses a stack to explore the graph as far as possible along each branch before backtracking
- Time Complexity: $O(n+m)$
- Space Complexity: $O(n)$

Topological Sort

- Linear ordering of vertices such that for every directed edge uv, vertex u comes before v in the ordering
- Can be performed using DFS
- Use linked list to store the topological order (add vertex to front of list after visiting all its neighbors)
- Time Complexity: $O(n+m)$
- Space Complexity: $O(n)$

SSSP - Relaxation of Nodes

- Set the source node's distance to 0, and all other nodes' distances to ∞.
- For each directed edge $u \to v$, if $dist[u] + weight(u,v) < dist[v]$, update $dist[v]$ to $dist[u] + weight(u,v)$.
- **Execution Strategy:**
  - **Bellman-Ford:** Repeat this relaxation for all edges until no updates can be made (maximum $n-1$ iterations).
  - **Dijkstra & DAG:** Process nodes systematically (greedy or topological order) so edges only need to be relaxed once per visited node.
- **Runtimes** (where $n = |V|$ and $m = |E|$):
  - **Bellman-Ford:** $O(nm)$
  - **DAG Shortest Path:** $O(n+m)$
  - **Dijkstra's Algorithm:** $O(m \log m)$ using lazy deletion, or $O((n+m)\log n)$ using a standard binary min-heap.

Dijkstra: (Lazy Deletion Version)

```
dijkstra(G, s):
    dist_est = [an array of all ∞]; set dist_est[s] = 0
    visited = [an array of all false]; heap = [a heap containing (0, s)]
    while heap is not empty:
        (distance, curr_node) = heap.extract_min();
        if visited[curr_node]: continue else visited[curr_node] = true
        dist_est[curr_node] = distance
        for all neighbours (n, w) in G.get_neighbours(curr_node)
            heap.insert(dist_est[curr_node] + w, n)
```

Union-Find Disjoint Sets (UFDS)

Maintains a collection of disjoint sets, allowing for highly efficient union and connectivity checks.

Core Arrays:

- `parent_array[x]`: Initially, every element is its own root (`parent_array[x] = x`).
- `rank[x]`: Initially, every element's rank is 0.

Union-by-rank:

```
union(x, y):
    x_root = find_root(x), y_root = find_root(y)
    if x_root == y_root: don't do anything, return :p
    if rank[y_root] < rank[x_root]:
        parent_array[y_root] = x_root
    if rank[x_root] < rank[y_root]:
        parent_array[x_root] = y_root
    if rank[x_root] == rank[y_root]:
        parent_array[x_root] = y_root, ++rank[y_root]
```

Time Complexity

- **Worst-Case Runtime:** $O(\log n)$ per operation.
- **Amortized Runtime:** $O(\alpha(n))$ per operation, where $\alpha(n)$ is the Inverse Ackermann function.
- **Practical Implication:** $\alpha(n) \leq 4$ for any realistically imaginable input size ($A(4,4) = 2^{2^{65536}} - 3$), meaning operations are effectively $O(1)$. A sequence of m operations takes practically $O(m)$ time.

Minimum Spanning Trees (MST)

- **Assumption:** Edge weights are bounded non-negative integers $w \in [0, c)$ for a constant c.

Faster Kruskal's Algorithm

- **Idea:** Replace the $O(m \log m)$ comparison-based sort with **Counting Sort**.
- **Time Complexity:** $O(c + m\alpha(n))$, where α is the inverse Ackermann function.

Faster Prim's Algorithm

- **Idea:** Replace the $O(\log n)$ Priority Queue with an array of c queues. To find the minimum edge, scan the array from index 0 to $c-1$ to find the first non-empty queue.
- **Time Complexity:** $O(c(n+m))$

Pseudocode:

```
in_tree = array of size n, initialized to false
fake_pq = array of c empty queues

in_tree[s] = true
for (n, weight) in get_neighbours(s):
    fake_pq[weight].push(n)

while fake_pq has a non-empty queue:
    curr_node = pop from the first non-empty queue in fal
    if in_tree[curr_node]:
        continue

    in_tree[curr_node] = true
    for (n, weight) in get_neighbours(curr_node):
        fake_pq[weight].push(n)
```

Appendix

sort	best	average	worst	stable?	me
bubble	$\Omega(n)$	$O(n^2)$	$O(n^2)$	✓	O
selection	$\Omega(n^2)$	$O(n^2)$	$O(n^2)$	×	O
insertion	$\Omega(n)$	$O(n^2)$	$O(n^2)$	✓	O
merge	$\Omega(n \log n)$	$O(n \log n)$	$O(n \log n)$	✓	O
quick	$\Omega(n \log n)$	$O(n \log n)$	$O(n^2)$	×	O
heap	$\Omega(n \log n)$	$O(n \log n)$	$O(n \log n)$	×	O

sorting invariants

sort	invariant (after k iterations)
bubble	largest k elements are sorted
selection	smallest k elements are sorted
insertion	first k slots are sorted
merge	given subarray is sorted
quick	partition is in the right position

searching

search	average
linear	$O(n)$
binary	$O(\log n)$
quickSelect	$O(n)$
interval	$O(\log n)$
all-overlaps	$O(k \log n)$
1D range	$O(k + \log n)$
2D range	$O(k + \log^2 n)$

data structures assuming $O(1)$ comparison cost

data structure	search	insert
sorted array	$O(\log n)$	$O(n)$
unsorted array	$O(n)$	$O(1)$
linked list	$O(n)$	$O(1)$
tree	$O(\log n)$ or $O(h)$	$O(\log n)$ or $O(h)$
trie	$O(L)$	$O(L)$
heap	$O(\log n)$ or $O(h)$	$O(\log n)$ or $O(h)$
dictionary	$O(\log n)$	$O(\log n)$
symbol table	$O(1)$	$O(1)$
chaining	$O(n)$	$O(1)$
open addressing	$\frac{1}{1-\alpha} = O(1)$	$O(1)$
priority queue	(contains) $O(1)$	$O(\log n)$
skip list	$O(\log n)$	$O(\log n)$

Orders of Growth

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < 2^{2n}$$
$$\log_a n < n^a < a^n < n! < n^n$$

Orders of Growth

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies O(n \log n)$$

$$T(n) = T\left(\frac{n}{2}\right) + O(n) \implies O(n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1) \implies O(n)$$

$$T(n) = T\left(\frac{n}{2}\right) + O(1) \implies O(\log n)$$

$$T(n) = 2T(n-1) + O(1) \implies O(2^n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n \log n) \implies O(n(\log n)^2)$$

$$T(n) = 2T\left(\frac{n}{4}\right) + O(1) \implies O(\sqrt{n})$$

$$T(n) = T(n-c) + O(n) \implies O(n^2)$$